FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Jaimin Judd D. Roces
Section:	AN43
Professor:	Ms. Tan
Date:	10/11/26

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation


# ----------------------------------------
# Get user input
# ----------------------------------------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))


# ----------------------------------------
# Agent Definition
# ----------------------------------------
class RandomWalker(ap.Agent):

    def setup(self):
        # Starting position of the agent
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )

        # Store the agent's movement history
        self.path = [self.position]

    def step(self):
        # Possible movement directions
        directions = [
            (1, 0),    # Right
            (-1, 0),   # Left
            (0, 1),    # Up
            (0, -1)   # Down
```

```
        ]

        # Shuffle directions so the agent still moves randomly
        random.shuffle(directions)

        old_position = self.position

        # Try each direction until a valid movement is found
        for direction in directions:

            new_x = old_position[0] + direction[0]
            new_y = old_position[1] + direction[1]

            # Check if the new position is inside the grid
            if (
                0 <= new_x < self.model.p.grid_size[0]
                and
                0 <= new_y < self.model.p.grid_size[1]
            ):

                new_position = (new_x, new_y)

                # ----------------------------------------
                # Task 2: Avoid other agents
                # ----------------------------------------
                occupied_positions = [
                    agent.position
                    for agent in self.model.agents
                    if agent != self
                ]

                # Move only if another agent is not occupying
                # the new position
                if new_position not in occupied_positions:
                    self.position = new_position
                    break

        # ----------------------------------------
        # Task 1: Track the agent's path
        # ----------------------------------------
        self.path.append(self.position)


# ----------------------------------------
# Model Definition
# ----------------------------------------
class RandomWalkModel(ap.Model):

    def setup(self):

        # Create the agents
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        # Create the grid
```

```
        self.grid = ap.Grid(
            self,
            self.p.grid_size,
            torus=False
        )

        # Add agents to the grid
        self.grid.add_agents(self.agents)


    def step(self):

        # Move every agent
        for agent in self.agents:
            agent.step()


# ----------------------------------------
# Parameters
# ----------------------------------------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}


# ----------------------------------------
# Create and setup the model
# ----------------------------------------
model = RandomWalkModel(parameters)
model.setup()


# ----------------------------------------
# Animation setup
# ----------------------------------------
fig, ax = plt.subplots()

ax.set_xlim(0, grid_size)
ax.set_ylim(0, grid_size)

ax.set_xticks(range(grid_size + 1))
ax.set_yticks(range(grid_size + 1))

ax.grid(True)

ax.set_title("Random Walk Simulation")


# Display agents as dots
scat = ax.scatter([], [], s=200)


# ----------------------------------------
# Update animation
# ----------------------------------------
def update(frame):
```

```
    # Move all agents
    model.step()

    # Get the x and y positions
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]

    # Update the dots
    scat.set_offsets(list(zip(x, y)))

    return scat,


# ----------------------------------------
# Run the animation
# ----------------------------------------
ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    blit=True,
    repeat=False
)

plt.show()


# ----------------------------------------
# Task 3: Display final positions
# ----------------------------------------
print("\n========== FINAL POSITIONS ==========")

for i, agent in enumerate(model.agents):
    print(
        f"Agent {i + 1}: "
        f"Final Position = {agent.position}"
    )


# ----------------------------------------
# Display paths
# ----------------------------------------
print("\n========== AGENT PATHS ==========")

for i, agent in enumerate(model.agents):

    print(f"\nAgent {i + 1} Path:")

    for step_number, position in enumerate(agent.path):
        print(
            f"Step {step_number}: {position}"
        )


# ----------------------------------------
# Task 3: Analyze final position distribution
# ----------------------------------------
```

```
print("\n========== POSITION ANALYSIS ==========")

position_count = {}

for agent in model.agents:

    position = agent.position

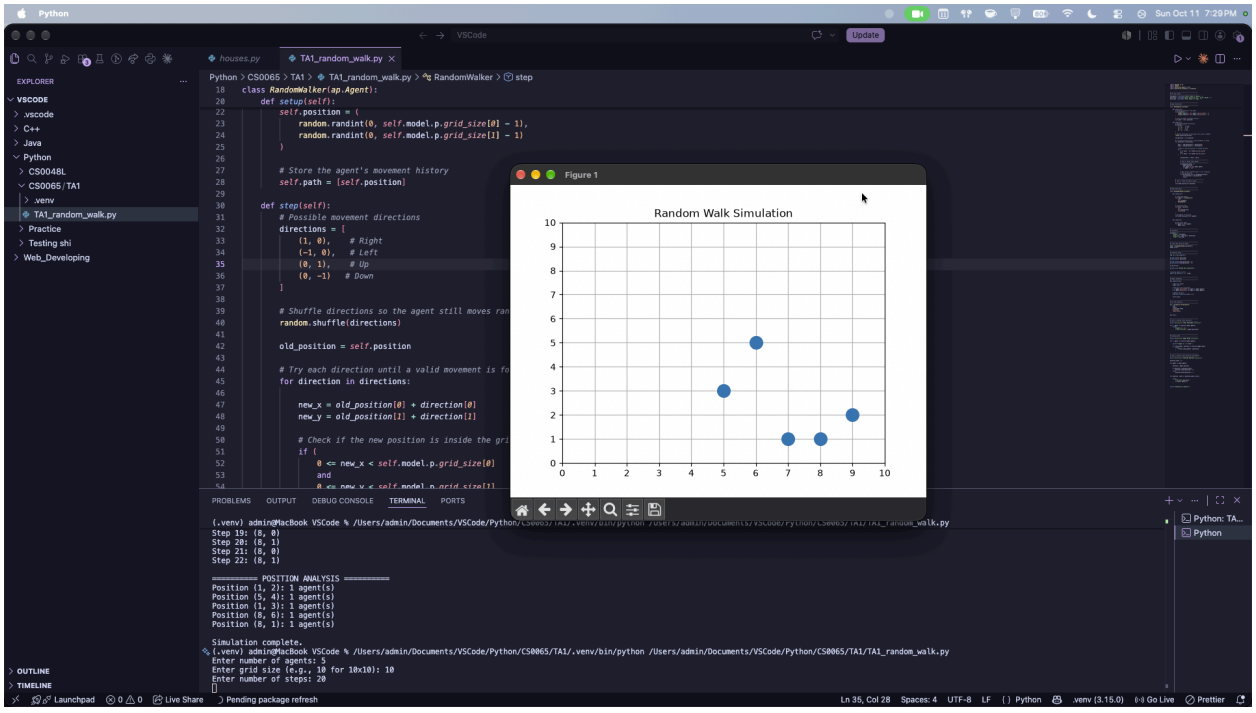
    if position in position_count:
        position_count[position] += 1
    else:
        position_count[position] = 1


for position, count in position_count.items():

    print(
        f"Position {position}: "
        f"{count} agent(s)"
    )


print("\nSimulation complete.")
```

5 agents, 10x10 grid size, 20 steps:



```
Simulation complete.
(.venv) admin@MacBook VSCode % /Users/admin/Documents/VSCode/Python/CS0065/TA1/.venv/bin/python /Users/admin/Documents/VSCode/Python/CS0065/TA1/TA1_random_walk.py
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20

========= FINAL POSITIONS =========
Agent 1: Final Position = (6, 5)
Agent 2: Final Position = (7, 1)
Agent 3: Final Position = (9, 2)
Agent 4: Final Position = (8, 1)
Agent 5: Final Position = (5, 3)

========= AGENT PATHS =========

Agent 1 Path:
Step 0: (8, 7)
Step 1: (9, 7)
Step 2: (9, 8)
Step 3: (9, 7)
Step 4: (8, 7)
Step 5: (7, 7)
Step 6: (6, 7)
Step 7: (6, 6)
Step 8: (5, 6)
Step 9: (4, 6)
Step 10: (5, 6)
Step 11: (6, 6)
Step 12: (6, 5)
Step 13: (6, 4)
Step 14: (6, 5)
Step 15: (5, 5)
Step 16: (4, 5)
Step 17: (4, 4)
Step 18: (4, 3)
Step 19: (4, 4)
Step 20: (5, 4)
Step 21: (5, 5)
Step 22: (6, 5)

Agent 2 Path:
Step 0: (0, 2)
Step 1: (0, 1)
Step 2: (0, 0)
Step 3: (0, 1)
Step 4: (0, 2)
Step 5: (0, 3)
Step 6: (0, 2)
Step 7: (1, 2)
Step 8: (1, 1)
Step 9: (2, 1)
Step 10: (2, 0)
Step 11: (3, 0)
Step 12: (2, 0)
Step 13: (3, 0)
Step 14: (2, 0)
Step 15: (3, 0)
Step 16: (3, 1)
Step 17: (3, 0)
Step 18: (4, 0)
Step 19: (5, 0)
Step 20: (5, 1)
```

```
Step 16: (8, 1)
Step 17: (8, 2)
Step 18: (9, 2)
Step 19: (9, 3)
Step 20: (9, 2)
Step 21: (9, 1)
Step 22: (9, 2)

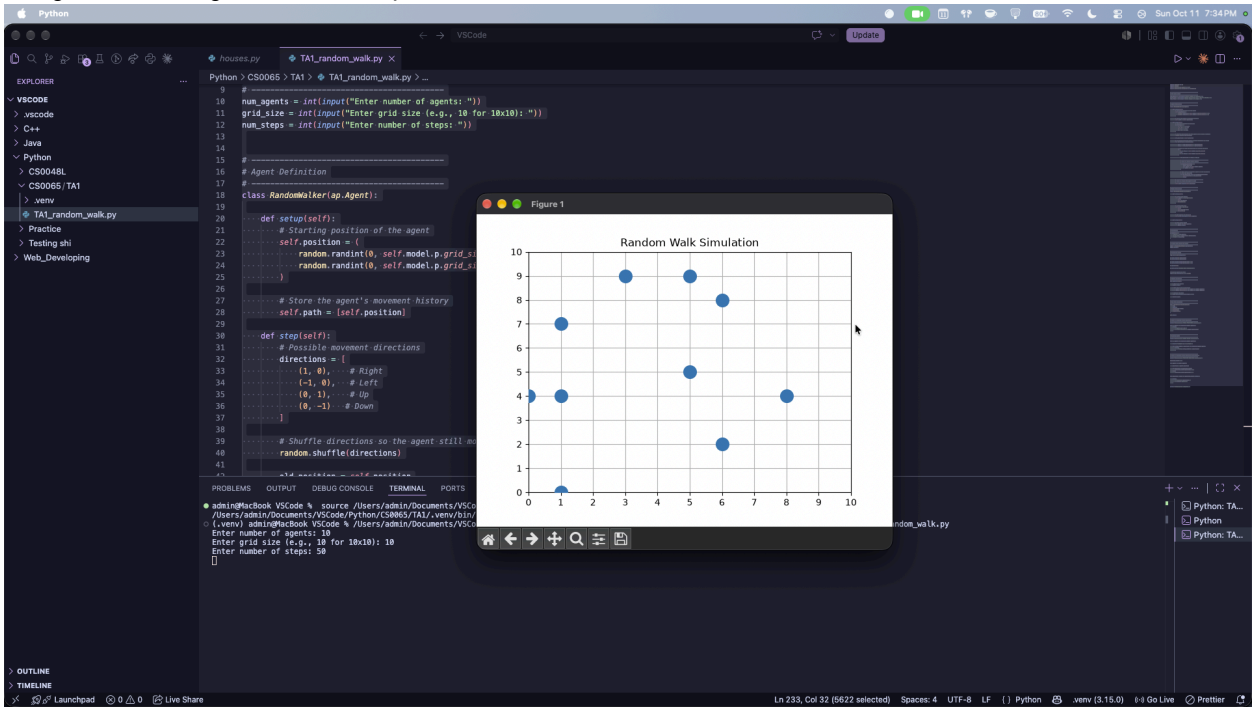
Agent 4 Path:
Step 0: (8, 1)
Step 1: (9, 1)
Step 2: (9, 2)
Step 3: (9, 3)
Step 4: (9, 2)
Step 5: (9, 1)
Step 6: (8, 1)
Step 7: (7, 1)
Step 8: (8, 1)
Step 9: (9, 1)
Step 10: (9, 2)
Step 11: (9, 3)
Step 12: (9, 4)
Step 13: (8, 4)
Step 14: (8, 5)
Step 15: (9, 5)
Step 16: (8, 5)
Step 17: (7, 5)
Step 18: (7, 4)
Step 19: (7, 3)
Step 20: (7, 2)
Step 21: (8, 2)
Step 22: (8, 1)

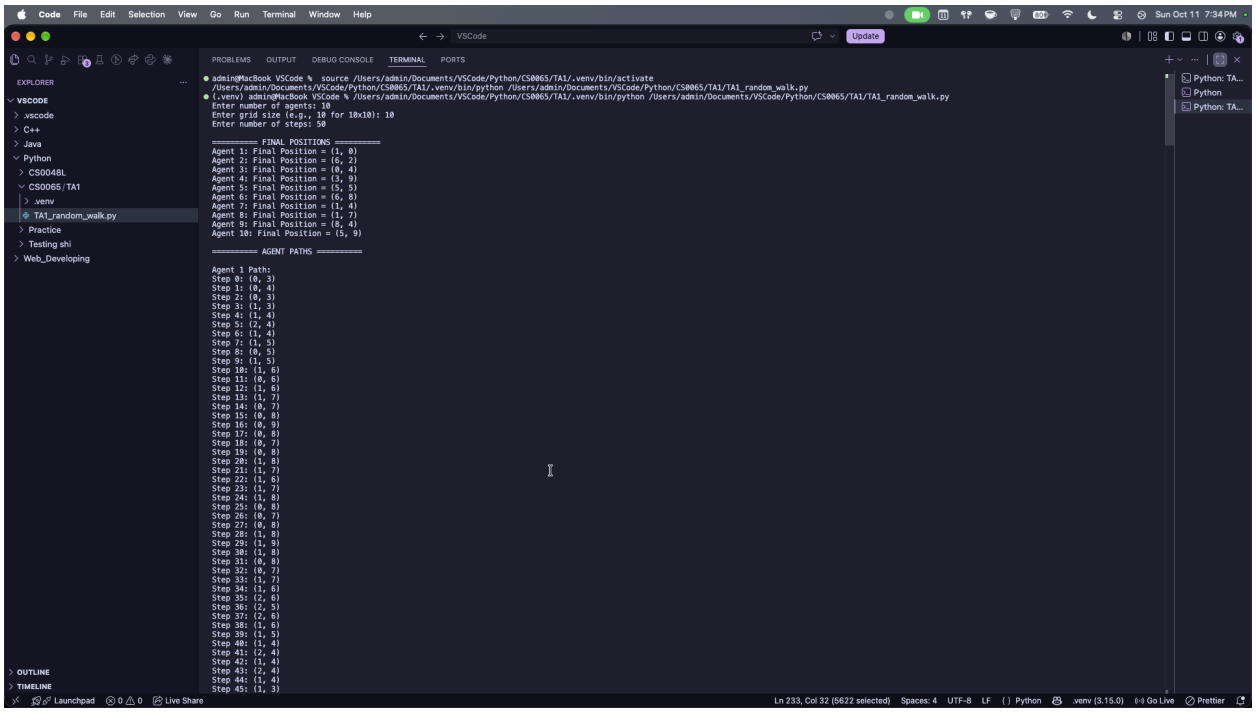
Agent 5 Path:
Step 0: (4, 4)
Step 1: (5, 4)
Step 2: (4, 4)
Step 3: (5, 4)
Step 4: (5, 3)
Step 5: (6, 3)
Step 6: (5, 3)
Step 7: (4, 3)
Step 8: (4, 4)
Step 9: (4, 3)
Step 10: (4, 2)
Step 11: (4, 1)
Step 12: (5, 1)
Step 13: (5, 2)
Step 14: (6, 2)
Step 15: (5, 2)
Step 16: (6, 2)
Step 17: (6, 3)
Step 18: (5, 3)
Step 19: (4, 3)
Step 20: (4, 4)
Step 21: (4, 3)
Step 22: (5, 3)

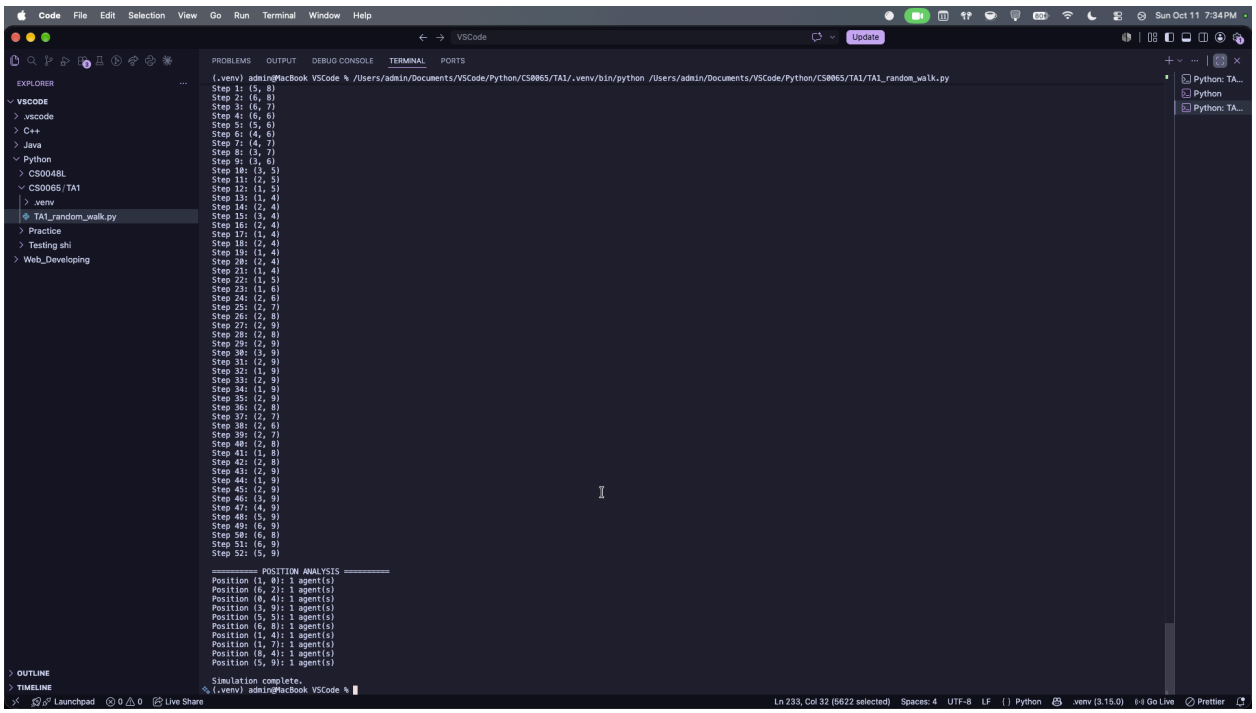
========== POSITION ANALYSIS ==========
Position (6, 5): 1 agent(s)
Position (7, 1): 1 agent(s)
Position (9, 2): 1 agent(s)
Position (8, 1): 1 agent(s)
Position (5, 3): 1 agent(s)

Simulation complete.
```

10 agents, 10x10 grid size, 50 steps:




Code File Edit Selection View Go Run Terminal Window Help
Sun Oct 11 7:34 PM
VSCode
Update
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
admin@MacBook VSCode % source /Users/admin/Documents/VSCode/Python/CS0065/TA1/.venv/bin/activate
/Users/admin/Documents/VSCode/Python/CS0065/TA1/.venv/bin/python /Users/admin/Documents/VSCode/Python/CS0065/TA1/TA1_random_walk.py
(.venv) admin@MacBook VSCode % /Users/admin/Documents/VSCode/Python/CS0065/TA1/.venv/bin/python /Users/admin/Documents/VSCode/Python/CS0065/TA1/TA1_random_walk.py
Enter number of agents: 10
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50
========= FINAL POSITIONS =========
Agent 1: Final Position = (1, 0)
Agent 2: Final Position = (6, 2)
Agent 3: Final Position = (0, 4)
Agent 4: Final Position = (3, 9)
Agent 5: Final Position = (5, 5)
Agent 6: Final Position = (6, 8)
Agent 7: Final Position = (1, 4)
Agent 8: Final Position = (1, 7)
Agent 9: Final Position = (0, 4)
Agent 10: Final Position = (5, 9)
========= AGENT PATHS =========
Agent 1 Path:
Step 0: (0, 3)
Step 1: (0, 4)
Step 2: (0, 3)
Step 3: (1, 3)
Step 4: (1, 4)
Step 5: (2, 4)
Step 6: (1, 4)
Step 7: (1, 5)
Step 8: (0, 5)
Step 9: (1, 5)
Step 10: (1, 6)
Step 11: (0, 6)
Step 12: (1, 6)
Step 13: (1, 7)
Step 14: (0, 7)
Step 15: (0, 8)
Step 16: (0, 9)
Step 17: (0, 8)
Step 18: (0, 7)
Step 19: (0, 8)
Step 20: (1, 8)
Step 21: (1, 7)
Step 22: (1, 6)
Step 23: (1, 7)
Step 24: (1, 8)
Step 25: (0, 8)
Step 26: (0, 7)
Step 27: (0, 8)
Step 28: (1, 8)
Step 29: (1, 9)
Step 30: (1, 8)
Step 31: (0, 8)
Step 32: (0, 7)
Step 33: (1, 7)
Step 34: (1, 6)
Step 35: (2, 6)
Step 36: (2, 5)
Step 37: (2, 6)
Step 38: (1, 6)
Step 39: (1, 5)
Step 40: (1, 4)
Step 41: (2, 4)
Step 42: (1, 4)
Step 43: (2, 4)
Step 44: (1, 4)
Step 45: (1, 3)
EXPLORER
VSCODE
.vscode
C++
Java
Python
CS0048L
CS0065 / TA1
.venv
TA1_random_walk.py
Practice
Testing shi
Web_Developing
OUTLINE
TIMELINE
Launchpad
Live Share
Ln 233, Col 32 (5622 selected)
Spaces: 4
UTF-8
LF
Python
.venv (3.15.0)
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun Oct 11 7:34 PM
VSCode
Update
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) admin@MacBook VSCode % /Users/admin/Documents/VSCode/Python/CS0065/TA1/.venv/bin/python /Users/admin/Documents/VSCode/Python/CS0065/TA1/TA1_random_walk.py
Step 1: (5, 8)
Step 2: (6, 8)
Step 3: (6, 7)
Step 4: (6, 6)
Step 5: (5, 6)
Step 6: (4, 6)
Step 7: (4, 7)
Step 8: (3, 7)
Step 9: (3, 6)
Step 10: (3, 5)
Step 11: (2, 5)
Step 12: (1, 5)
Step 13: (1, 4)
Step 14: (2, 4)
Step 15: (3, 4)
Step 16: (2, 4)
Step 17: (1, 4)
Step 18: (2, 4)
Step 19: (1, 4)
Step 20: (2, 4)
Step 21: (1, 4)
Step 22: (1, 5)
Step 23: (1, 6)
Step 24: (2, 6)
Step 25: (2, 7)
Step 26: (2, 8)
Step 27: (2, 9)
Step 28: (2, 8)
Step 29: (2, 9)
Step 30: (3, 9)
Step 31: (2, 9)
Step 32: (1, 9)
Step 33: (2, 9)
Step 34: (1, 9)
Step 35: (2, 9)
Step 36: (2, 8)
Step 37: (2, 7)
Step 38: (2, 6)
Step 39: (2, 7)
Step 40: (2, 8)
Step 41: (1, 8)
Step 42: (2, 8)
Step 43: (2, 9)
Step 44: (1, 9)
Step 45: (2, 9)
Step 46: (3, 9)
Step 47: (4, 9)
Step 48: (5, 9)
Step 49: (6, 9)
Step 50: (6, 8)
Step 51: (6, 9)
Step 52: (5, 9)
========= POSITION ANALYSIS =========
Position (1, 0): 1 agent(s)
Position (6, 2): 1 agent(s)
Position (0, 4): 1 agent(s)
Position (3, 9): 1 agent(s)
Position (5, 5): 1 agent(s)
Position (6, 8): 1 agent(s)
Position (1, 4): 1 agent(s)
Position (1, 7): 1 agent(s)
Position (0, 4): 1 agent(s)
Position (5, 9): 1 agent(s)
Simulation complete.
(.venv) admin@MacBook VSCode %
EXPLORER
VSCODE
.vscode
C++
Java
Python
CS0048L
CS0065 / TA1
.venv
TA1_random_walk.py
Practice
Testing shi
Web_Developing
OUTLINE
TIMELINE
Launchpad
Live Share
Ln 233, Col 32 (5622 selected)
Spaces: 4
UTF-8
LF
Python
.venv (3.15.0)
Go Live
Prettier

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. If the number of agents is very high compared to the grid size, the agents become more crowded and there is less space for them to move freely. Since our modified simulation makes agents avoid each other, they may have more difficulty finding an unoccupied position. This can result in more agents staying in the same general areas or having fewer available movement options.
2. If the number of agents is very high compared to the grid size, the agents become more crowded and there is less space for them to move freely. Since our modified simulation makes agents avoid each other, they may have more difficulty finding an unoccupied position. This can result in more agents staying in the same general areas or having fewer available movement options.
3. No, the agents do not move outside the grid. The program checks whether a new position is within the grid boundaries before allowing an agent to move there. The original code specifically limits the x and y positions between the minimum and maximum grid values.
4. This simulation can be used to model situations where objects or people move randomly or change positions over time. For example, it can represent diffusion, where particles spread from one area to another, or crowd movement, where people move around a limited space. By changing the number of agents, grid size, and number of steps, different real-life situations can be simulated and observed.